

Pentru a evalua capacitatea AI-ului de a interactiona cu DOM-ul aplicatiei, am solicitat Google Gemini generarea unui test E2E, cu urmatorul prompt:

"Scrie un test automat in Python folosind Playwright pentru site-ul demo.nopcommerce.com. Testul trebuie să caute cuvantul 'laptop', sa sorteze rezultatele după pret descrescător (High to Low) si sa valideze printr-un assert ca pretul primului produs este mai mare sau egal cu pretul celui de-al doilea produs."

Rezultatul:

Scrie un test automat in Python folosind Playwright pentru site-ul demo.nopcommerce.com. Testul trebuie să caute cuvantul 'laptop', sa sorteze rezultatele după pret descrescător (High to Lo...

◆ Arată procesul de gândire

lată codul în Python folosind Playwright. Scriptul utilizează API-ul sincron pentru simplitate și acoperă exact pașii ceruți: navigare, căutare, sortare, parsarea textului pentru a extrage valorile numerice și aserțiunea finală.

Scriptul de test

Python

```
from playwright.sync_api import sync_playwright
import re

def test_sortare_pret_descrescator():
    with sync_playwright() as p:
        # Rulăm în mod non-headless pentru a putea observa execuția
        browser = p.chromium.launch(headless=False)
        context = browser.new_context()
```

Raspunsul complet:

lată codul în Python folosind Playwright. Scriptul utilizează API-ul sincron pentru simplitate și acoperă exact pașii ceruți: navigare, căutare, sortare, parsarea textului pentru a extrage valorile numerice și aserțiunea finală.

Scriptul de test

Python

```
from playwright.sync_api import sync_playwright
import re
```

```
def test_sortare_pret_descrescator():
    with sync_playwright() as p:
        # Rulăm în mod non-headless pentru a putea observa execuția
        browser = p.chromium.launch(headless=False)
```

```

context = browser.new_context()
page = context.new_page()

# 1. Navigare pe site
page.goto("https://demo.nopcommerce.com/")

# 2. Căutarea cuvântului 'laptop'
page.fill("#small-searchterms", "laptop")
page.click("button.search-box-button")

# Așteptăm ca rezultatele să se încarce
page.wait_for_selector(".product-item")

# 3. Sortarea rezultatelor
# Selectăm opțiunea folosind label-ul vizibil din dropdown
page.select_option("#products-orderby", label="Price: High to Low")

# Sortarea declanșează o reîncărcare a paginii/rezultatelor.
# networkidle asigură că noile rezultate au fost aduse de la
server.
page.wait_for_load_state("networkidle")
page.wait_for_selector(".product-item")

# 4. Extragerea și formatarea prețurilor
# Luăm toate elementele care conțin prețul efectiv
elemente_pret = page.locator(".actual-price").all_inner_texts()

# Validare de siguranță: testul nu are sens dacă nu există minim 2
produse
assert len(elemente_pret) >= 2, "Test eșuat: Nu s-au găsit cel
puțin două produse pentru a putea compara prețurile."

def curata_pret(pret_text):
    # Eliminăm simbolurile valutare ($, €, etc) și virgulele
    folosite pentru mii
    pret_curat = re.sub(r'^\d.', '', pret_text)
    return float(pret_curat)

preturi = [curata_pret(pret) for pret in elemente_pret]

# 5. Aserțiunea cerută
assert preturi[0] >= preturi[1], f"Aserțiune eșuată: Primul preț
({preturi[0]}) nu este mai mare sau egal cu al doilea ({preturi[1]})."

print("Test trecut: Primul produs este mai scump sau egal cu al
doilea.")

```

```
browser.close()

if __name__ == "__main__":
    test_sortare_pret_descrescator()
```

Critici asupra abordării cerute

Deși scriptul de mai sus rezolvă exact ceea ce ai cerut, abordarea în sine are câteva probleme fundamentale din perspectiva testării automate profesionale:

1. **Testezi backend-ul prin frontend:** Sortarea datelor este responsabilitatea bazei de date și a API-ului. Să validezi logica de sortare printr-un test E2E (End-to-End) pe interfața grafică este inefficient și lent. Acest tip de verificare trebuie făcut la nivel de API/Unit Test. În UI, ar trebui doar să verifici dacă dropdown-ul declanșează un request valid.
2. **Fragilitatea datelor de test (Flakiness):** Testul presupune implicit că termenul "laptop" va returna mereu cel puțin două produse. Dacă mâine cineva șterge produsele de pe site-ul demo, testul tău va crăpa, dar nu din cauza unui bug în cod, ci din cauza lipsei datelor de test.
3. **Parsarea string-urilor:** UI-ul afișează prețuri formate localizat (ex: **\$1,500.00** sau **1.500,00 €**). Logica de curățare cu Regex devine un coșmar de mentenanță dacă site-ul își schimbă setările de monedă sau regiune.
4. **Dependența de timpii de încărcare:** Folosirea **networkidle** este funcțională, dar poate duce la teste lente. Într-un mediu CI/CD real, o abordare mai strictă ar fi să aștepți ca URL-ul să conțină parametrul de sortare (ex: **orderby=11**) și ca un anumit produs cheie să apară în DOM.

Dupa ce am rulat codul generat, nu am putut trece nici macar de verificarea Cloudflare.

Diferentele dintre codul generat si suita proprie de teste:

1. AI-ul nu a luat in calcul prezenta Cloudflare, codul generat executand o navigare directa folosind metoda `page.goto()`, urmata imediat de o incercare de interactiune cu formularul de cautare. Aceasta secventa declanseaza instantaneu o eroare de tip `TimeoutError`, deoarece platforma intercepteaza cererea si afiseaza pagina de verificare, care nu poate fi evitata. Testele scrise de noi utilizeaza extensia `playwright-stealth` si instructiuni de asteptare pentru a trece cu succes de filtrele de protectie.
2. Dupa ce sorteaza produsele, AI-ul foloseste comanda `networkidle` pentru a astepta ca produsele sa se incarce. Daca site-ul incarca reclame sau scripturi in fundal, testul va ramane blocat si va pica. In suita noastra de teste, am asteptat exact raspunsul de la server dupa ce am dat click (`page.expect_response()`).
3. Apar foarte des mesaje verzi de succes care acopera alte butoane de pe ecran. AI-ul nu este constient de ele si incearca sa dea click "prin" ele, ceea ce duce la erori. Noi a trebuit sa scriem cod specific pentru a inchide aceste ferestre inainte sa trecem la pasul urmator.

Concluzii:

Chiar daca AI-ul nu a reusit sa genereze un test functional, acesta ne-a ajutat din alte puncte de vedere: a sugerat modificarea arhitecturii proiectului si sa nu folosim date hardcodate (asa cum le folosim atunci cand completam adrese de livrare in timpul testelor).

Exemplu practic de utilizare AI pentru imbunatatirea proiectului:

Page Object Model (POM) este un design pattern care separa logica testelor si structura paginii web, utilizand clase. Fiecare pagina a site-ului devine o clasa separata in cod si contine proprietati (selectorii CSS, ID-urile sau elementele XPath utilizate pentru a gasi butoanele, campurile text si link-urile) si metode (functii care descriu comportamentele utilizatorului pe acea pagina, de exemplu: autentificare, adauga in cos, aplica cupon, etc.). "Stratul" de teste contine doar rularea scenariilor si assert-urile.

Am putea folosi un prompt pentru a solicita AI-ului sa implementeze acest design pattern, codul actual fiind vulnerabil la eventuale modificari de UX ale site-ului de test (selectorii si pasii de testare nu sunt separati, deci orice modificare minora de interfata ar necesita rescrierea a zeci de linii de cod).

Deci, am putea utiliza AI-ul pentru a imbunatati calitatea si sustenabilitatea codului din proiect.